SOFTWARE REQUIREMENTS SPECIFICATION

Process Automation and Order Tracking System
(PAOTS)

Prepared by:
Tasan, Carmina
Talla, Van Rexel
Pilapil, Trisha Jane

Course: Software Engineering
Instructor: Joseph Jaymel Morpos
Institution: Eastern Visayas State University – EVSU
Date: February 22, 2026

TABLE OF CONTENTS

1. INTRODUCTION
   1.1 Purpose
   1.2 Scope
   1.3 Definitions, Acronyms, and Abbreviations
   1.4 References
   1.5 Document Overview
2. OVERALL DESCRIPTION
   2.1 Product Perspective
   2.2 Product Features
   2.3 User Roles and Characteristics
   2.4 Operating Environment
   2.5 Design Constraints
   2.6 Assumptions and Dependencies
3. USE CASE SCENARIOS
   3.1 UC-01: Create New Order (Staff)
   3.2 UC-02: Update Production Status (Designer)
   3.3 UC-03: Collect Order (Staff)
   3.4 UC-04: Search Order (Staff)
   3.5 UC-05: Process Payment (Staff)
   3.6 UC-06: Generate Claim Stub (Staff)
4. USER STORIES
5. FUNCTIONAL REQUIREMENTS
6. NON-FUNCTIONAL REQUIREMENTS
7. SYSTEM ARCHITECTURE
8. INTERFACE REQUIREMENTS
9. DATA REQUIREMENTS
10. ERROR HANDLING REQUIREMENTS
11. SECURITY REQUIREMENTS
12. CONCURRENCY REQUIREMENTS
13. ACCEPTANCE CRITERIA
14. APPENDICES
   Appendix A – Use Case Diagram
   Appendix B – Entity-Relationship Diagram
   Appendix C – System Architecture Diagram
   Appendix D – Requirements Traceability Matrix

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Process Automation and Order Tracking System (PAOTS). The document is intended for the development team, project stakeholders, and academic evaluators. PAOTS is designed to serve as a real-time digital logbook for a small digital printing shop — replacing manual, paper-based order tracking with a centralized web-based system that ensures no order is lost and all staff always know the current status of every job.

1.2 Scope

PAOTS is a web-based application for internal use by staff and designers at a single printing shop. The system covers the full order lifecycle: customer intake by counter staff, production status tracking by the designer, and order collection. The system does not automate the physical work — designing, printing, and cutting are always performed by humans. The system's role is to track that work and make the information visible to everyone in real time.

The system is explicitly OUT OF SCOPE for the following:

- Online customer portals or customer-facing order submission
- Automated control of physical printing equipment
- Payroll, HR, or supplier management
- Multi-branch operations (Phase 1 only)

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
PAOTS	Process Automation and Order Tracking System
SRS	Software Requirements Specification
GUI	Graphical User Interface
Tracking ID	Unique alphanumeric identifier assigned to each order upon creation
Claim Stub	Printed or on-screen document given to the customer as proof of order
RBAC	Role-Based Access Control
TLS	Transport Layer Security
HTTPS	Hypertext Transfer Protocol Secure
SQL	Structured Query Language
XSS	Cross-Site Scripting
API	Application Programming Interface
FR	Functional Requirement
NFR	Non-Functional Requirement
UC	Use Case
US	User Story
RTM	Requirements Traceability Matrix
Supabase	Open-source backend platform providing PostgreSQL database, file storage, and authentication as a service

1.4 References

- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications
- Software Engineering Course Guidelines, EVSU
- Printing Shop Operational Workflow (internal documentation)
- OWASP Top 10 Web Application Security Risks (2021)
- Supabase Documentation – https://supabase.com/docs

1.5 Document Overview

Section 2 provides an overall system description. Sections 3 and 4 define use case scenarios and user stories. Sections 5 and 6 enumerate functional and non-functional requirements. Sections 7 through 9 describe the system architecture, interfaces, and data model. Sections 10 through 12 address error handling, security, and concurrency. Section 13 defines acceptance criteria. Section 14 contains appendices including diagrams and the Requirements Traceability Matrix.

2. OVERALL DESCRIPTION

2.1 Product Perspective

PAOTS is a standalone web-based application for a single small printing shop. It replaces handwritten order slips, verbal status updates between staff and the designer, and the risk of lost or misattributed files. The system is best understood as a shared digital logbook: staff create entries at the counter, the designer updates those entries as work progresses, and everyone can see the current state of every order at any time.

The system is built on a modern, cloud-capable stack. Supabase serves as the unified backend, providing PostgreSQL database storage, file storage for customer layout files, and user authentication — eliminating the need for a physical local server or on-site network-attached storage hardware. The React frontend runs in any modern browser with no installation required.

2.2 Product Features

- Role-based login: Staff and Designer accounts with restricted access per role
- Digital order entry form with automatic Tracking ID generation
- File upload for customer layout files, stored in Supabase Storage
- Automatic pricing calculation based on item type, dimensions, and quantity
- Real-time production status board visible to all users simultaneously
- Designer queue showing all orders currently awaiting design work
- Payment tracking: downpayment recording and balance-due display
- On-screen claim stub with Tracking ID for the customer
- Order search by customer name or Tracking ID
- Manager dashboard with daily sales summary and order statistics
- Inventory threshold alerts for consumable materials
- Audit log of all status changes and payment events

2.3 User Roles and Characteristics

PAOTS has three primary roles. In a small shop setting, the Designer and Printer Operator are typically the same person — the system treats them as a single "Designer" role. A Manager role is available for reporting and configuration access.

Role	Who They Are	What They Do in the System	System Access
Staff	Counter staff / sales person	Creates orders, uploads files, records payments, searches orders, marks Collected	Order entry, payment, search, claim stub
Designer	Graphic artist / printer operator (may be one person)	Views design queue, downloads layout files, updates production status (Designing → Printing → Ready)	Designer queue, status updates only

Manager	Shop owner or supervisor	Views reports, monitors inventory, manages user accounts	Full read access, reports, configuration

2.4 Operating Environment

- Web browser: Google Chrome (latest) or Microsoft Edge (latest) — no plugins required
- Client devices: Desktop or laptop computers; tablet support is a future consideration
- Backend: Supabase (cloud-hosted PostgreSQL database, file storage, and authentication)
- Frontend hosting: Static hosting service (e.g., Vercel, Netlify, or GitHub Pages)
- Optional peripheral: Standard desktop or inkjet printer for claim stub printout
- Network: Internet connection required for Supabase access

2.5 Design Constraints

- Must support layout file uploads in formats: .pdf, .psd, .jpg, .png
- File storage is handled by Supabase Storage — no physical NAS hardware is required
- Thermal receipt printer integration is deferred to a future phase; Phase 1 uses a browser-printable claim stub
- The system must function entirely in the browser with no desktop application installation
- All data transmissions must use HTTPS

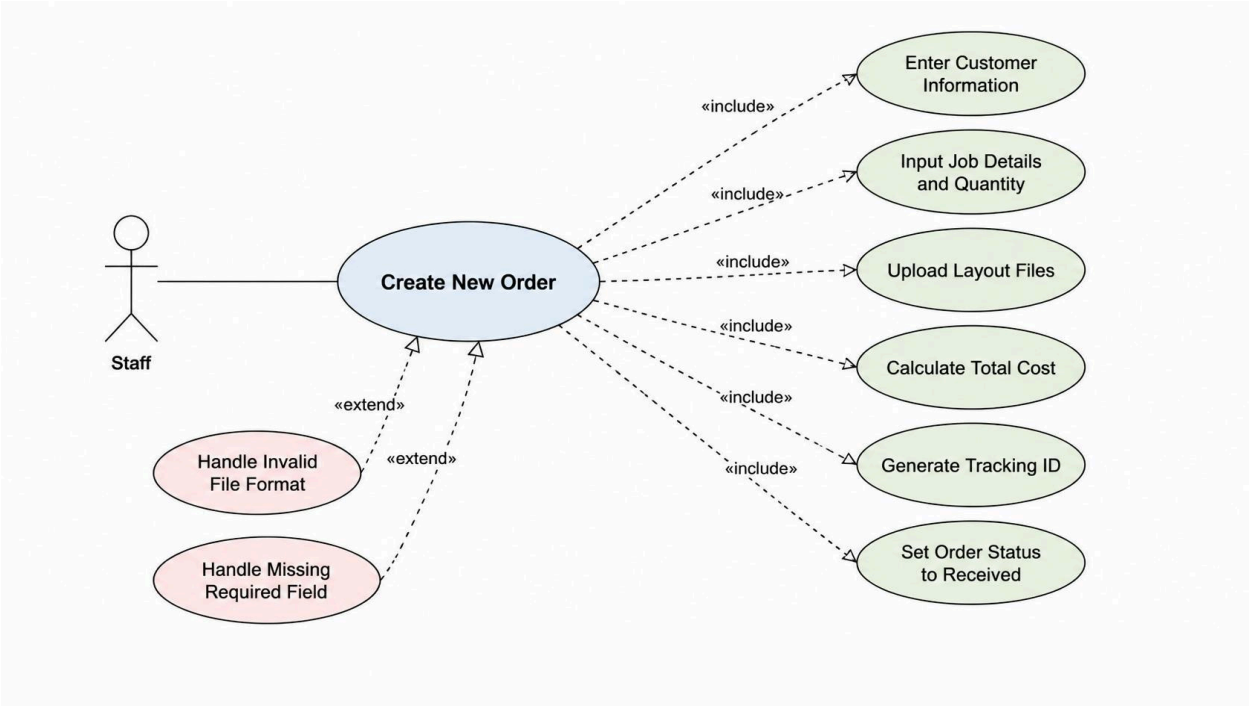
2.6 Assumptions and Dependencies

- Shop computers have a reliable internet connection to access Supabase
- Supabase free or paid tier is provisioned and configured before deployment
- Staff members have basic computer and browser literacy
- The designer's computer is connected to the same internet connection and can access the system
- Physical printing work (designing in Photoshop/Illustrator, operating the printer and cutter) is performed outside the system

3. USE CASE SCENARIOS

3.1 UC-01: Create New Order

Field	Detail
Actor	Staff
Description	A customer arrives at the counter. Staff enters the order details into the system, uploads the layout file, and confirms the order.
Preconditions	Staff is logged in. The customer has provided their name, contact number, and layout file.
Trigger	Staff selects "New Order" from the dashboard.



Main Success Scenario

1. Staff selects "New Order" from the counter dashboard.
2. Staff enters the customer's name and contact number. The system checks for an existing customer profile with the same name and contact number and pre-fills if found.
3. Staff selects the job type (e.g., tarpaulin, jersey, intra-board, sticker) and enters the specifications: dimensions, material type, and quantity.
4. Staff uploads the customer's layout file (.pdf, .psd, .jpg, or .png). The system renames the file automatically using the pattern: CustomerName_TrackingID_originalfilename and stores it in Supabase Storage.
5. The system calculates and displays the total cost based on the job type, dimensions, and quantity.

6. Staff selects the payment type (Full Payment or Downpayment) and payment method (Cash or E-wallet), then enters the amount received.
7. Staff confirms the order. The system generates a unique alphanumeric Tracking ID, sets the order status to "Received," saves the order to the database, and displays the claim stub on screen.
8. Staff prints the claim stub on a standard printer (or writes the Tracking ID down) and hands it to the customer.

Alternative Flows

A1 – Invalid File Format: If the uploaded file is not in a supported format (.pdf, .psd, .jpg, .png), the system displays an inline error message and rejects the upload. The order cannot be confirmed until a valid file is attached or the upload is explicitly marked as "File to be submitted later."

A2 – Missing Required Fields: If the staff member attempts to confirm the order without completing all mandatory fields (customer name, contact number, item type, quantity), the system highlights the incomplete fields with a red border and prevents submission.

A3 – Pricing Engine Incomplete: If the system cannot compute a price because dimensions or material type have not been entered, it displays a "Cannot calculate — please enter all specifications" notice and prevents confirmation until the information is complete.

A4 – Duplicate Customer Detected: If an existing customer profile matches the entered name and contact number, the system offers to link the new order to that existing profile rather than creating a duplicate record.

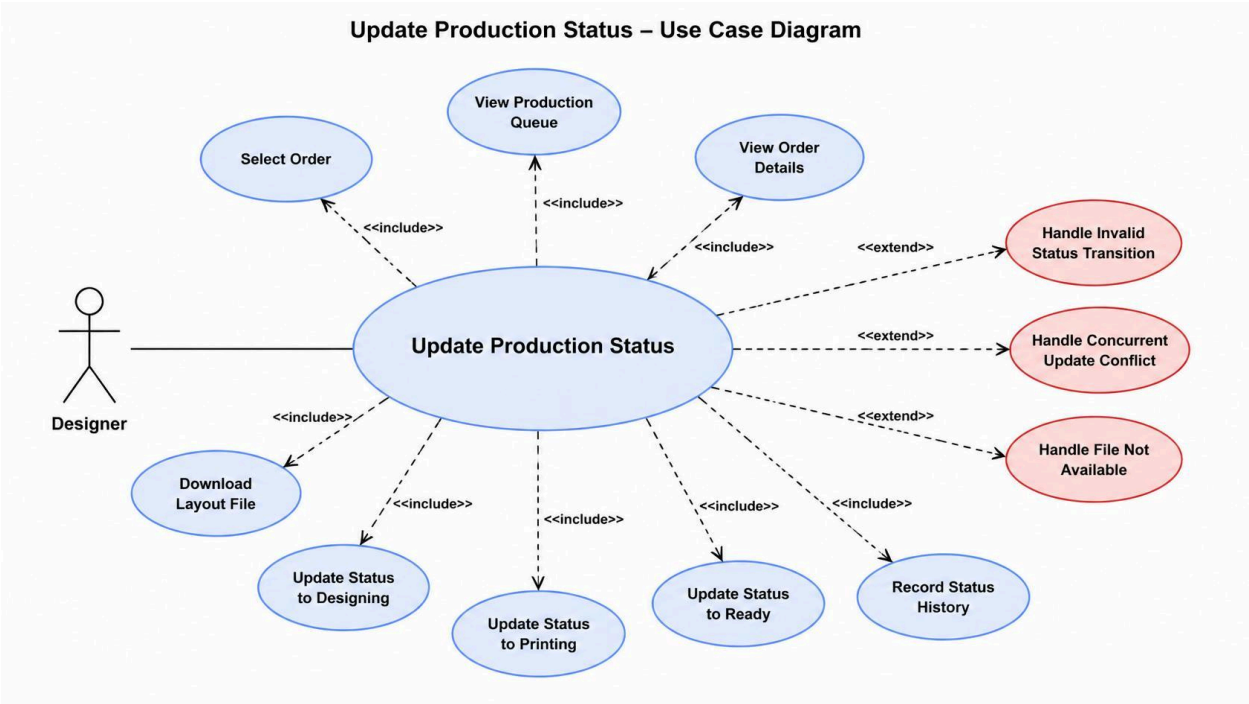
Postconditions

- A new order record exists in the database with a unique Tracking ID and status "Received."
- The layout file is stored in Supabase Storage and linked to the order record.
- A payment record is created reflecting the amount paid and the remaining balance.
- The claim stub is displayed on screen for printing.
- The order appears in the designer's queue.

3.2 UC-02: Update Production Status (Designer)

Field	Detail
Actor	Designer
Description	The designer opens the system on their workstation, sees the queue of pending orders, selects one, downloads the layout file, does the physical design work externally in Photoshop or Illustrator, then returns to the system and advances the order's status.
Preconditions	Designer is logged in. At least one order exists with status "Received" or higher.

Trigger	Designer opens the dashboard and sees pending work in their queue.



Main Success Scenario

9. Designer logs in and sees the production queue, showing all orders sorted by creation date (oldest first).
10. Designer selects an order to work on. The system displays the order details: job type, dimensions, material type, and the customer's uploaded layout file.
11. Designer clicks "Download File" to retrieve the layout file to their local workstation.
12. Designer performs the design work externally in Photoshop, Illustrator, or equivalent software. The system is not involved in this step.
13. Designer returns to the system and updates the order status to "Designing." The system records the transition in the Status History log.
14. When the job is sent to the physical printer, the designer updates the status to "Printing."
15. When the printed item has been checked and is ready for collection, the designer updates the status to "Ready." The system records the transition.

Alternative Flows

A1 – Invalid Status Transition: If the designer attempts to skip a status (e.g., jump from "Received" directly to "Ready"), the system rejects the action with a validation message listing the expected next status. Status must be advanced one step at a time.

A2 – Concurrent Update Conflict: If two users attempt to update the same order status simultaneously, the first submitted update succeeds. The second user receives a "This order was just updated — please refresh" notification and must review the current status before proceeding.

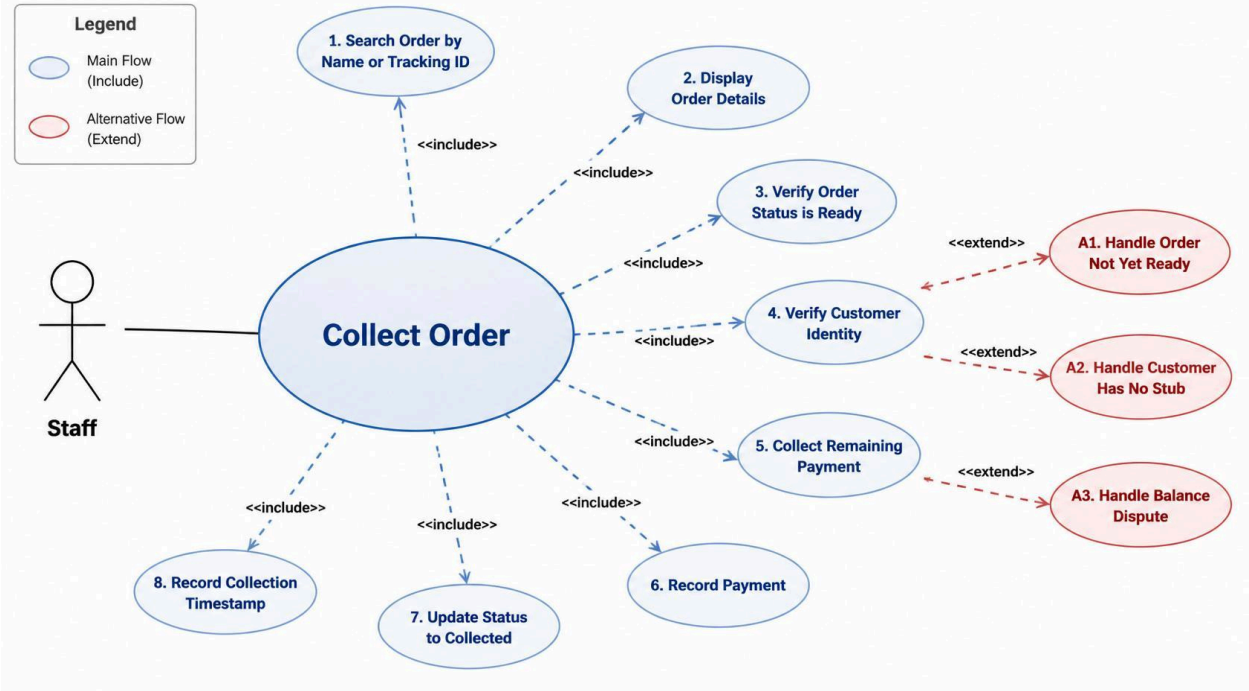
A3 – File Not Yet Available: If the designer clicks on an order that has no layout file attached (marked "File to be submitted later"), the system displays a "No file available" notice. The designer may proceed with the status update only if they have received the file through another channel.

Postconditions

- The order status is updated and the change is recorded in the Status History table with a timestamp and the designer's user ID.
- The updated status is visible in real time on all connected devices.
- When status becomes "Ready," the staff dashboard displays the order in the "Ready for Collection" section.

3.3 UC-03: Collect Order (Staff)

Field	Detail
Actor	Staff
Description	The customer returns to the shop to collect their finished order. Staff retrieves the order, verifies the status is "Ready," collects any remaining balance, and marks the order as "Collected."
Preconditions	Staff is logged in. The order status is "Ready."
Trigger	A customer arrives at the counter with their claim stub or provides their name.



Main Success Scenario

16. Staff searches for the order by customer name or Tracking ID.

17. System displays the order details, confirming that the current status is "Ready."
18. Staff verifies the customer's identity against the order record.
19. If a balance is outstanding, staff collects the remaining payment, selects the payment method, and records it in the system.
20. Staff updates the order status to "Collected."
21. The system records the final status change and the collection timestamp.

Alternative Flows

A1 – Order Not Yet Ready: If the searched order's status is not "Ready" (e.g., still "Printing"), the system displays the current status clearly. Staff informs the customer of the delay. The order is not marked "Collected."

A2 – Customer Has No Stub: Staff uses the search by customer name to retrieve the order. If multiple orders share the same name, the system lists them with dates and status, and staff selects the correct one.

A3 – Balance Dispute: If the remaining balance shown in the system does not match the customer's understanding, staff must resolve the discrepancy manually. A manager may void and re-record the payment entry if an error is identified.

Postconditions

- The order status is updated to "Collected" with a timestamp.
- Any final payment is recorded in the payment history.
- The order is closed and no longer appears in the active production queue.

3.4 UC-04: Search Order

Field	Detail
Actor	Staff or Manager
Description	A staff member retrieves an existing order record using the customer's name or Tracking ID.
Preconditions	At least one order exists in the database.
Trigger	Staff enters a search term in the search field.

Main Success Scenario

22. Staff enters the customer's name or Tracking ID in the search field.
23. The system queries the database and returns matching results within 2 seconds.
24. Staff selects the correct order from the results list.
25. The system displays the full order details: customer information, job specifications, current status, payment history, and the attached file reference.

Alternative Flows

A1 – No Results: If no records match the search term, the system displays "No orders found" and suggests checking the spelling or trying a partial name. No error is raised.

A2 – Multiple Matches: If multiple orders share the same customer name, the system lists all matching records sorted by most recent creation date. Each result shows the Tracking ID, status, and creation date to help staff identify the correct order.

Postconditions

- The matching order record is displayed for the staff member to view or act upon.

3.5 UC-05: Process Payment

Field	Detail
Actor	Staff
Description	Staff records a payment transaction against an existing order — either at order creation (downpayment) or at order collection (balance payment).
Preconditions	The order exists and has a recorded total cost.
Trigger	Customer presents payment at the counter.

Main Success Scenario

26. Staff opens the order record and selects "Record Payment."
27. The system displays the total cost, amount already paid, and the remaining balance.
28. Staff enters the payment amount and selects the payment method (Cash or E-wallet).
29. The system validates that the amount entered does not exceed the remaining balance.
30. The system records the transaction and updates the payment status: if the full balance is cleared, status updates to "Paid Full"; otherwise it remains "Partial / Downpayment."

Alternative Flows

A1 – Overpayment Entered: If the staff member enters an amount exceeding the remaining balance, the system displays a validation error specifying the maximum acceptable amount. The transaction is not recorded until a valid amount is entered.

A2 – Payment Correction (Manager): If a payment was recorded in error, a Manager may mark the transaction as voided from the payment history. The void event is logged with the manager's user ID, a timestamp, and a reason note.

Postconditions

- The payment transaction is recorded in the Payment table.
- The order's balance-due value is updated.
- Payment status reflects the new financial position of the order.

3.6 UC-06: Generate Claim Stub

Field	Detail
Actor	Staff
Description	The system displays an on-screen claim stub for the customer after order creation. Staff prints this on a standard printer or the customer photographs the Tracking ID from the screen.
Preconditions	Order has been confirmed and a Tracking ID has been generated.
Trigger	Order creation is successfully completed (automatic, immediately after UC-01 Step 7).

Note: A thermal receipt printer is not required in Phase 1. The claim stub is a browser-printable page containing the customer's name, Tracking ID, order summary, amount paid, balance due, and estimated pickup date. In shops that do not have a printer at the counter, staff may read the Tracking ID aloud or the customer photographs it from the screen.

Main Success Scenario

31. Immediately after order confirmation, the system navigates to the claim stub view.
32. The claim stub displays: customer name, Tracking ID, job type and quantity, total cost, amount paid, balance due, and estimated pickup date.
33. Staff selects "Print" to send the stub to any connected printer using the browser's standard print dialog.
34. The customer receives the physical printout or photographs the screen.

Alternative Flows

A1 – No Printer Available: Staff reads the Tracking ID aloud and the customer photographs the screen. The claim stub remains accessible at any time by searching for the order.

A2 – Reprint Request: Staff may reprint the stub at any time by opening the order record and selecting "View Claim Stub." Reprints are not separately tracked but the order record serves as the source of truth.

Postconditions

- The customer has their Tracking ID and can return to collect their order.

4. USER STORIES

4.1 US-01

User Story: As a Staff member, I want to input a customer order quickly at the counter so that the queue moves efficiently during busy hours.

Acceptance Criteria:

- Order entry form is completable within 2 minutes under normal conditions.
- Required fields are visually marked; the form prevents submission if any are empty.
- The system auto-generates the Tracking ID — staff do not type it manually.

4.2 US-02

User Story: As a Staff member, I want to upload the customer's layout file when creating their order so that the file stays linked to that specific order and is never lost or confused with another.

Acceptance Criteria:

- Supported upload formats: .pdf, .psd, .jpg, .png.
- The stored filename is automatically prefixed with the customer name and Tracking ID.
- Files exceeding the configured size limit are rejected with a clear error message.
- The file link is visible on the order detail page at all times.

4.3 US-03

User Story: As a Designer, I want to see a queue of all orders waiting for my attention so that I know exactly what to work on next without asking the counter staff.

Acceptance Criteria:

- The designer dashboard shows all orders with status "Received" sorted by creation date (oldest first).
- Each queue entry shows: customer name, job type, dimensions, and file availability.
- The queue updates in real time when new orders are added by staff.

4.4 US-04

User Story: As a Designer, I want to update the status of an order as I progress through the work so that everyone in the shop always knows the current state of every job.

Acceptance Criteria:

- Allowed status transitions: Received → Designing → Printing → Ready.
- Each transition is saved with a timestamp and the designer's user ID.
- The updated status is visible on all connected devices within 2 seconds.

4.5 US-05

User Story: As a Staff member, I want to search for an order by customer name or Tracking ID so that I can find any order instantly, even if the customer has lost their claim stub.

Acceptance Criteria:

- Search results appear in under 2 seconds.
- Results show current status, payment balance, and Tracking ID.
- Partial name search is supported.

4.6 US-06

User Story: As a Staff member, I want to see the remaining balance clearly when a customer comes to collect so that I collect the correct amount without manual calculation.

Acceptance Criteria:

- The order detail view shows: total cost, amount paid, and balance due.
- Balance due is displayed prominently on the collection screen.
- Recording a final payment updates the balance to zero and flags the order as "Paid Full."

4.7 US-07

User Story: As a Manager, I want to view a daily sales summary so that I can track how much revenue the shop collected today.

Acceptance Criteria:

- The manager dashboard shows total orders created today and total payments collected today.
- Summary is filterable by date.
- Report is exportable to PDF or Excel.

4.8 US-08

User Story: As a Staff member, I want to give the customer a claim stub with their Tracking ID so that they have a reference for when they return to collect their order.

Acceptance Criteria:

- After order confirmation, the system displays a printable claim stub.
- The stub includes: customer name, Tracking ID, total cost, amount paid, balance due, and estimated pickup date.
- The stub is printable using the browser's standard print function on any connected printer.

5. FUNCTIONAL REQUIREMENTS

5.1 FR-01: User Authentication and Role-Based Access

Description: The system shall provide secure login and enforce role-based access for Staff, Designer, and Manager accounts.

Rationale: Role separation ensures staff cannot accidentally (or intentionally) alter production statuses, and designers cannot touch payment data.

Acceptance Criteria:

- Valid credentials are required to access any part of the system.
- Each role sees only the modules and actions relevant to their function: Staff sees order entry, search, and payment; Designer sees the production queue and status controls only; Manager sees all of the above plus reports and configuration.
- Accounts are locked after five (5) consecutive failed login attempts.
- Sessions expire automatically after 30 minutes of inactivity.
- Authentication is handled by Supabase Auth.

5.2 FR-02: Order Entry and Tracking ID Generation

Description: The system shall provide a digital order form to capture all required customer and job details at the point of sale, and shall generate a unique Tracking ID upon confirmation.

Rationale: Replaces handwritten order slips and eliminates the risk of lost or illegible records.

Acceptance Criteria:

- Mandatory fields: customer name, contact number, job type, dimensions, material type, quantity.
- The system prevents form submission if any required field is empty.
- Upon confirmation, the system generates a unique alphanumeric Tracking ID automatically. Staff do not type or choose this value.
- The order is immediately saved to the Supabase database with status "Received."
- The new order appears in the designer's queue within 2 seconds of confirmation.

5.3 FR-03: Layout File Upload and Storage

Description: The system shall allow staff to upload the customer's layout file and link it to the order.

Rationale: Ensures the design file is never lost, misattributed to another customer, or inaccessible when the designer needs it.

Acceptance Criteria:

- Supported formats: .pdf, .psd, .jpg, .png.
- Files are stored in Supabase Storage. No physical NAS hardware is required.

- The stored filename is automatically generated in the format: CustomerName_TrackingID_originalfilename.
- Files exceeding 200MB are rejected with a clear error message. Files below this limit must upload without corruption.
- A file integrity check (checksum comparison) is performed on upload to detect corrupted transfers.
- The designer can download the file directly from the order detail view.

5.4 FR-04: Production Status Workflow

Description: The system shall track each order through a defined set of production statuses and record every transition.

Rationale: Gives every person in the shop a real-time view of what is happening with every order, without needing to ask anyone.

Acceptance Criteria:

- Defined status sequence: Received → Designing → Printing → Ready → Collected.
- Staff may only set statuses "Received" (automatic on creation) and "Collected" (at pickup).
- Designers may only advance statuses from "Received" through to "Ready."
- Status transitions must follow the defined sequence; skipping steps is not permitted.
- Each status change is recorded in the Status History table with a timestamp and the user ID of the person who made the change.
- Status updates are reflected in real time across all connected devices.

5.5 FR-05: Pricing Engine

Description: The system shall automatically calculate the total order cost based on job type, dimensions, material type, and quantity.

Rationale: Eliminates manual pricing errors and ensures consistent, auditable cost computation.

Acceptance Criteria:

- Price calculations are accurate to two (2) decimal places.
- Applicable taxes and fees are applied as configured in the system settings.
- The calculated price is displayed before the staff member confirms the order.
- If required pricing inputs are missing, the system prevents order confirmation and displays a specific error.

5.6 FR-06: Payment Management

Description: The system shall record, display, and manage the complete payment history of every order, including downpayments, balance payments, and voided transactions.

Rationale: Prevents revenue loss through missed balance collection and provides an auditable payment record.

Acceptance Criteria:

- Payment statuses: Downpayment, Partial, Paid Full.
- The order detail view shows: total cost, total paid to date, and remaining balance — clearly and prominently.
- Payment method is recorded for each transaction (Cash or E-wallet).
- Overpayment is rejected with a validation error specifying the maximum acceptable amount.
- Managers may void erroneous payment records; all void events are logged with user ID, timestamp, and reason.
- Payment history for any order is accessible from the order detail view.

5.7 FR-07: Order Search

Description: The system shall allow staff to retrieve any order by customer name or Tracking ID.

Rationale: Enables staff to locate an order immediately, even when the customer has lost their claim stub.

Acceptance Criteria:

- Search accepts customer name (partial match supported) or exact Tracking ID.
- Results are returned in under 2 seconds.
- Each result displays: customer name, Tracking ID, current status, payment balance, and creation date.
- If no records match, the system displays a clear "No orders found" message.

5.8 FR-08: Claim Stub Display and Print

Description: After order confirmation, the system shall display an on-screen claim stub that staff can print using any connected printer.

Rationale: Gives the customer a physical or photographable reference for order collection, without requiring specialized hardware.

Acceptance Criteria:

- The claim stub displays: customer name, Tracking ID, job summary, total cost, amount paid, balance due, and estimated pickup date.
- The claim stub is printable using the browser's standard print function.
- The stub is accessible at any time from the order detail view for reprinting.
- Thermal receipt printer hardware is not required in Phase 1.

5.9 FR-09: Inventory Alerts

Description: The system shall monitor consumable stock levels and alert the manager when any item falls below a configurable threshold.

Rationale: Prevents production delays caused by running out of materials unexpectedly.

Acceptance Criteria:

- Managers can set minimum threshold levels for each material type (e.g., tarpaulin roll, jersey fabric, ink).
- An alert is displayed on the manager dashboard when stock falls at or below the threshold.
- Alerts persist until the manager updates the inventory level.

5.10 FR-10: Reporting and Dashboard

Description: The system shall provide a manager dashboard displaying daily operational metrics and a sales summary.

Rationale: Gives the manager an accurate, real-time picture of the shop's activity without manual tallying.

Acceptance Criteria:

- Dashboard shows: total orders created today, orders by status, total payments collected today.
- The daily sales figure matches the sum of all payment transactions recorded for the selected date.
- Reports are filterable by date range.
- Reports are exportable to PDF and Excel.
- Dashboard data refreshes automatically every 60 seconds.

6. NON-FUNCTIONAL REQUIREMENTS

6.1 Performance

Search results shall return within 2 seconds for a database of up to 500 active order records. Order creation form submission shall complete within 3 seconds. Dashboard data shall refresh within 5 seconds. These thresholds assume a standard internet connection of 10 Mbps or higher.

6.2 Security

All transmissions shall use HTTPS/TLS 1.2 or higher. Passwords shall be stored as salted hashes (Supabase Auth uses bcrypt by default). Role-based access control is enforced at both the frontend routing and backend API levels. Sessions expire after 30 minutes of inactivity. All database queries use parameterized statements to prevent SQL injection. Input rendered in the browser is escaped to prevent XSS. Uploaded files are validated for format and size before storage. Account lockout is enforced after five consecutive failed login attempts. All critical events are recorded in the audit log.

6.3 Usability

Staff shall be able to complete a full order entry within 2 minutes. Error messages shall be written in plain language and indicate exactly what the user must do to resolve the issue. The interface shall be readable in a brightly lit shop environment, using high-contrast colors and sufficiently large text. No system training beyond basic computer literacy shall be required for Staff or Designer roles.

6.4 Reliability

The system shall maintain 99% uptime during defined operating hours (dependent on Supabase service availability). Supabase provides automated daily backups. In the event of a connection interruption, in-progress form data shall be preserved in the browser until the session ends, and the user shall be notified of the connectivity issue.

6.5 Scalability

The system shall support up to 500 order transactions per day without performance degradation. The data model and architecture shall support future expansion to additional product types, user accounts, or a second branch without requiring a full redesign.

6.6 Maintainability

Source code shall include inline documentation for all custom business logic components (pricing engine, Tracking ID generation, status transition validation). The codebase shall follow a consistent folder structure and naming convention documented in a README file.

6.7 Accuracy

All financial calculations (total cost, taxes, balance due, payments) shall be accurate to two (2) decimal places. Amounts displayed on the claim stub shall exactly match values stored in the database.

6.8 Compatibility

The system shall function correctly on Google Chrome (latest stable) and Microsoft Edge (latest stable) on Windows 10 or later. No browser extensions, plugins, or desktop application installations shall be required.

7. SYSTEM ARCHITECTURE

PAOTS follows a three-tier architecture. The React frontend communicates with a Node.js/Express backend for business logic operations, and Supabase serves as the unified data and storage layer. This design eliminates the need for a physical local server or on-site hardware.

7.1 Technology Stack

Layer	Component	Technology
Presentation	Staff and Designer browser interface	React.js
Business Logic	REST API — pricing engine, Tracking ID generation, status validation	Node.js + Express
Data	Relational database (PostgreSQL)	Supabase (hosted)
File Storage	Customer layout files	Supabase Storage
Authentication	User login and session management	Supabase Auth
Hosting	Frontend static hosting	Vercel / Netlify / GitHub Pages
Transport	Client-server communication	HTTPS / TLS 1.2+

7.2 Architecture Notes

Because Supabase provides a client-side JavaScript library, simple read operations (such as loading the order list or the designer queue) can be performed by React calling Supabase directly, without passing through the Express server. The Express backend is used specifically for operations that require server-side business logic: pricing calculation (to prevent price manipulation from the client), Tracking ID generation, and status transition validation. This reduces backend complexity while keeping critical logic secure.

Figure 1. System Architecture Diagram — see Appendix C.

8. INTERFACE REQUIREMENTS

8.1 User Interface

- The Staff dashboard shall present: a "New Order" button, a search field, and a list of today's active orders with their current status.
- The Designer dashboard shall present: a production queue sorted by creation date, with each entry showing the job type, dimensions, and a "Download File" button.
- Both dashboards shall use high-contrast colors and font sizes appropriate for a brightly lit shop environment.
- All forms shall display inline validation errors without requiring a page reload.
- Status updates shall appear on the dashboard in real time without manual page refresh.

8.2 Software Interface

- The React frontend communicates with the Express backend via RESTful HTTP endpoints.
- The React frontend communicates with Supabase directly for read operations using the Supabase JavaScript client library.
- The Express backend communicates with Supabase using the Supabase service role key for privileged operations.
- The optional notification integration (email or SMS) connects to a third-party messaging provider via its API.

8.3 Hardware Interface

- The system uses the browser's standard print dialog to send the claim stub to any printer connected to the staff computer. No specialized print driver or thermal printer hardware is required in Phase 1.
- The system has no direct interface with physical printing or cutting equipment. All machine operation is performed manually by shop staff outside the system.

9. DATA REQUIREMENTS

9.1 Database Overview

The following entities form the relational schema stored in Supabase (PostgreSQL). All tables use surrogate UUID primary keys generated by the database.

Order

id (PK), customer_id (FK), created_by (FK→User), job_type, dimensions, material_type, quantity, total_cost, downpayment_amount, balance_due, status, estimated_pickup_date, created_at, updated_at

Customer

id (PK), name, contact_number, email, created_at

User

id (PK), username, role (staff | designer | manager), is_active, last_login, created_at — passwords managed by Supabase Auth

Payment

id (PK), order_id (FK), amount, payment_method (cash | ewallet), payment_status (downpayment | partial | paid_full), recorded_by (FK→User), transaction_date, is_voided, voided_by (FK→User), void_reason, voided_at

FileAttachment

id (PK), order_id (FK), original_filename, stored_filename, storage_path, file_size_bytes, file_format, uploaded_by (FK→User), uploaded_at, checksum

StatusHistory

id (PK), order_id (FK), old_status, new_status, changed_by (FK→User), changed_at

Inventory

id (PK), material_type, current_stock, threshold_level, unit, last_updated

AuditLog

id (PK), user_id (FK), action, target_table, target_id, details, created_at

9.2 Data Integrity Rules

- Every Order must have a valid customer_id and created_by (staff user ID) before being saved.
- Tracking IDs (the "id" field displayed to users) must be unique across all orders.
- All financial figures (total_cost, downpayment_amount, balance_due, payment amounts) are stored and displayed to exactly two decimal places.

- Files larger than 200MB must not be stored. All stored file paths must reference files that physically exist in Supabase Storage.
- Status transitions recorded in StatusHistory must conform to the valid workflow sequence.
- Voided payment records retain is_voided = TRUE and are never hard-deleted (soft delete only).

9.3 Backup and Recovery

- Supabase performs automated daily database backups on paid plans; the project shall use at minimum the Pro tier for production deployment.
- Point-in-time recovery (PITR) shall be enabled to allow restoration to any moment within the retention window.
- Supabase Storage files are replicated and managed by Supabase infrastructure.
- The development team shall document a recovery runbook specifying the steps to restore the database and verify data integrity after an incident.

10. ERROR HANDLING REQUIREMENTS

The following error conditions shall be explicitly handled by the system. For each condition, the system shall display a clear, plain-language message and provide a specific action for the user to take. The system shall never display raw database errors or stack traces to end users.

EH-01: Failed or Corrupted File Upload

If a file upload fails during transfer or fails the post-upload checksum verification, the system rejects the file, displays a "File upload failed — please try again" message, and discards the partial upload from Supabase Storage. The order cannot be confirmed until the upload succeeds or is explicitly marked "File pending."

EH-02: Unsupported File Format

If an uploaded file's extension or detected MIME type is not in the supported list (.pdf, .psd, .jpg, .png), the system displays "This file type is not supported. Please upload a .pdf, .psd, .jpg, or .png file" and prevents the file from being stored.

EH-03: File Too Large

If an uploaded file exceeds 200MB, the system displays "File exceeds the 200MB limit. Please compress or resize the file before uploading" and rejects the upload.

EH-04: Payment Overpayment

If the staff member enters a payment amount exceeding the remaining balance, the system displays "Amount entered exceeds the outstanding balance of [Php X.XX]. Please enter a valid amount" and prevents the transaction from being recorded.

EH-05: Invalid Status Transition

If a user attempts to set a status that violates the defined workflow sequence (e.g., jumping from "Received" to "Ready"), the system displays "Invalid status change. The next valid status is [X]" and rejects the transition.

EH-06: Concurrent Order Update Conflict

If two users update the same order simultaneously, the first committed change succeeds. The second user receives "This order was updated by someone else just now. Please refresh to see the latest status before continuing."

EH-07: Session Expired

If the user's session expires due to 30 minutes of inactivity, the system redirects to the login page. Any in-progress order form data is preserved in the browser's local state and recoverable within the same browser session if the user logs back in promptly.

EH-08: Supabase Connection Failure

If the system cannot reach Supabase (e.g., internet outage), the interface displays "Unable to connect to the server. Please check your internet connection and try again." No data is written or displayed until connectivity is restored.

EH-09: No File Attached to Order

If the designer opens an order that has no layout file attached, the system displays "No file attached to this order — check with the counter staff." The designer may still advance the status if they have received the file through another means.

EH-10: Order Not Found During Search

If a search returns no results, the system displays "No orders found matching [search term]" without raising an error state. Staff are prompted to check the spelling or try a Tracking ID search.

11. SECURITY REQUIREMENTS

SEC-01: Transport Security

All communications between the browser and the server (React → Express → Supabase) shall use HTTPS with TLS 1.2 or higher. Unencrypted HTTP requests shall be redirected to HTTPS automatically.

SEC-02: Password Storage

User passwords are stored as salted hashes by Supabase Auth (bcrypt). Plaintext passwords shall never be stored, logged, or transmitted by any component of the system.

SEC-03: Role-Based Access Control (RBAC)

Module and data access shall be enforced at both the frontend routing level and the Express API level for custom operations. Supabase Row Level Security (RLS) policies shall restrict database access at the data layer: Staff may read and write only orders they created; Designers may read all orders but write only status fields; Managers have full read access. No role may access data beyond what their function requires.

SEC-04: Session Management

Sessions managed by Supabase Auth shall expire after 30 minutes of inactivity. Session tokens are stored in HTTP-only, Secure cookies or Supabase's managed token storage, not in plain localStorage.

SEC-05: SQL Injection Prevention

All database interactions shall use Supabase's parameterized query API or the Supabase client library, which generates parameterized queries by default. Raw SQL string concatenation with user input is prohibited.

SEC-06: Cross-Site Scripting (XSS) Prevention

React's default JSX rendering escapes all user-supplied content before display. Staff-entered data (customer names, job descriptions) shall never be rendered as raw HTML. Content Security Policy (CSP) headers shall be configured on the hosting platform.

SEC-07: File Upload Security

Uploaded files shall be validated for extension and MIME type before storage. Files are stored in Supabase Storage with access controlled by RLS policies — files are not publicly accessible without an authenticated signed URL. Files are stored outside any public static directory.

SEC-08: Account Lockout

Accounts shall be locked after five (5) consecutive failed login attempts within a 15-minute window. Lockout is enforced by Supabase Auth configuration. Locked accounts may be unlocked by a Manager or via an email-based reset.

SEC-09: Audit Logging

The system shall write a record to the AuditLog table for all critical operations: logins, order creation and modification, status transitions, payment recording, void events, file uploads, and account management actions. Each log entry records: user_id, action, target_table, target_id, timestamp.

12. CONCURRENCY REQUIREMENTS

PAOTS is used simultaneously by at least two types of users (Staff at the counter, Designer at their workstation) and potentially by a Manager reviewing reports. The following requirements govern how the system behaves when multiple users interact with shared data at the same time.

CON-01: Status Update Conflict Resolution

If two users attempt to update the same order's status simultaneously, the system commits the first received update and informs the second user that the record was modified. The second user must refresh the order before proceeding. No order shall be left in an undefined or invalid status.

CON-02: Status Transition Atomicity

Each status transition shall be executed as a single atomic database operation. If the operation fails midway, the system rolls back to the previous valid status. No partial status writes are permitted.

CON-03: Payment Recording Atomicity

Recording a payment (including updating the order's balance_due field) shall be executed as a single atomic transaction. Concurrent payment attempts against the same order shall be serialized by the database to prevent race conditions in balance calculation.

CON-04: Real-Time Dashboard Consistency

Dashboard displays (order queues, status boards) shall reflect changes made by any user within 2 seconds, through either polling or Supabase's real-time subscription feature. If the connection to Supabase is interrupted, the interface shall display a "Connection lost — data may be outdated" banner until connectivity is restored.

13. ACCEPTANCE CRITERIA

The system shall be considered ready for production use when all of the following conditions have been verified by the development team and confirmed by the project stakeholders.

13.1 Functional Acceptance

- An order can be created by Staff, assigned a unique Tracking ID, and immediately appears in the designer's queue.
- The designer can open an order, download the attached layout file, and advance the status through the full sequence (Received → Designing → Printing → Ready).
- Staff can search for any order by customer name or Tracking ID and retrieve the correct record in under 2 seconds.
- Staff can mark an order as "Collected" and record the final payment. The order's balance updates to zero and status to "Collected."
- File uploads in all supported formats (.pdf, .psd, .jpg, .png) are stored successfully with the correct renamed format.
- Files with unsupported formats or exceeding 200MB are rejected with a specific error message.
- Overpayment amounts are rejected with a validation error.
- A claim stub displaying all required information is printable using the browser's print dialog.
- The manager daily sales report matches the sum of all payment transactions for the selected date.
- Account lockout activates after five consecutive failed login attempts.

13.2 Non-Functional Acceptance

- Search results return in under 2 seconds with a database of at least 500 order records.
- Simultaneous status updates by two users trigger the conflict notification for the second user without corrupting the order record.
- All data transmissions occur over HTTPS — no plaintext HTTP connections are accepted.
- A user with the Designer role cannot access the payment recording module.
- A user with the Staff role cannot access the Manager reports module.
- Sessions expire after 30 minutes of inactivity and redirect to the login page.

13.3 Stakeholder Sign-Off

- All functional requirements (FR-01 through FR-10) have been tested and verified.
- All identified error conditions (EH-01 through EH-10) produce the expected user-facing messages.
- Stakeholder (shop owner or instructor) approval is documented before production deployment.

14. APPENDICES

Appendix A – Use Case Diagram

The Use Case Diagram illustrates the interactions between the three user roles — Staff, Designer, and Manager — and the system use cases defined in Section 3. Key actor-to-use-case relationships:

- Staff: Create New Order, Search Order, Process Payment, Collect Order, Generate Claim Stub
- Designer: Update Production Status (Designing, Printing, Ready)
- Manager: View Reports, Manage Inventory, Manage User Accounts
- Both Staff and Manager: Search Order

[Insert Use Case Diagram — UML diagram showing actors and use case relationships]

Appendix B – Entity-Relationship Diagram (ERD)

The ERD defines the relational structure of the PAOTS database. Key relationships:

- Customer (1) → Order (N): One customer may have multiple orders.
- Order (1) → Payment (N): One order may have multiple payment transactions.
- Order (1) → FileAttachment (N): One order may have one or more attached files.
- Order (1) → StatusHistory (N): Every status transition creates one status history record.
- User (1) → Order (N): One staff user may create many orders.
- User (1) → AuditLog (N): Every user action generates audit log entries.

[Insert finalized ER Diagram — visual entity-relationship diagram]

Appendix C – System Architecture Diagram

The System Architecture Diagram illustrates the three-tier structure: the React frontend (browser-based, hosted on Vercel/Netlify), the Node.js/Express backend API (handles pricing, Tracking ID generation, and status validation), and the Supabase layer (PostgreSQL database, file storage, and authentication). All connections between layers use HTTPS. The React frontend also communicates directly with Supabase for simple read operations using the Supabase client library, bypassing Express for those calls.

[Insert Architecture Diagram — three-tier architecture diagram]

Appendix D – Requirements Traceability Matrix (RTM)

The RTM maps each Functional Requirement to its corresponding Use Cases and User Stories to confirm full requirements coverage.

FR ID	Requirement Name	Related Use Cases	Related User Stories	Key Acceptance Criteria
FR-01	User Authentication & RBAC	UC-01 to UC-06	All user stories require login	Login succeeds with valid credentials; lockout after 5

				failures; role restrictions enforced
FR-02	Order Entry & Tracking ID	UC-01	US-01	Order saved with unique Tracking ID; required fields validated; appears in designer queue within 2 sec
FR-03	File Upload & Storage	UC-01, UC-02	US-02	Supported formats stored with correct filename; files >200MB rejected; checksum validated
FR-04	Production Status Workflow	UC-02, UC-03	US-03, US-04	Valid sequence enforced; each change logged with user ID and timestamp; real-time update
FR-05	Pricing Engine	UC-01	US-01	Cost accurate to 2 decimal places; missing inputs blocked
FR-06	Payment Management	UC-01, UC-03, UC-05	US-06	Balance computed and displayed; overpayment rejected; void events logged
FR-07	Order Search	UC-04	US-05	Results within 2 sec; partial name match works; no-results message shown
FR-08	Claim Stub Display & Print	UC-06	US-08	Stub displays all required fields; printable via browser print dialog
FR-09	Inventory Alerts	—	US-07	Dashboard alert shown when stock at or below threshold
FR-10	Reporting & Dashboard	—	US-07	Sales total matches sum of payments; export to PDF and Excel works